Day 120: Hierarchical Clustering From Scratch, With a Real Dendrogram

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 18, Day 2 | Complete study guide

Watch: Hierarchical clustering - explained (TileStats) | Hierarchical Clustering & Dendrograms — Single, Complete, Average & Ward Linkage (Naive Statistician)

Script: `day120_hierarchical_clustering.py` · every number in this guide is copied by program from `day120_run_output.txt`, the real output of that script.

k-means needs k in advance. Agglomerative clustering builds a whole tree of merges instead, and any k can be cut from it afterwards. Four linkages are written from scratch with one update rule, checked against SciPy, and compared on three real datasets. Ward linkage wins everywhere (0.6153, 0.7899, 0.7940), and on digits it beats Day 119's k-means (0.6525). Single linkage collapses into one giant cluster: cut into 10 groups, one holds 1788 of 1797 digits.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

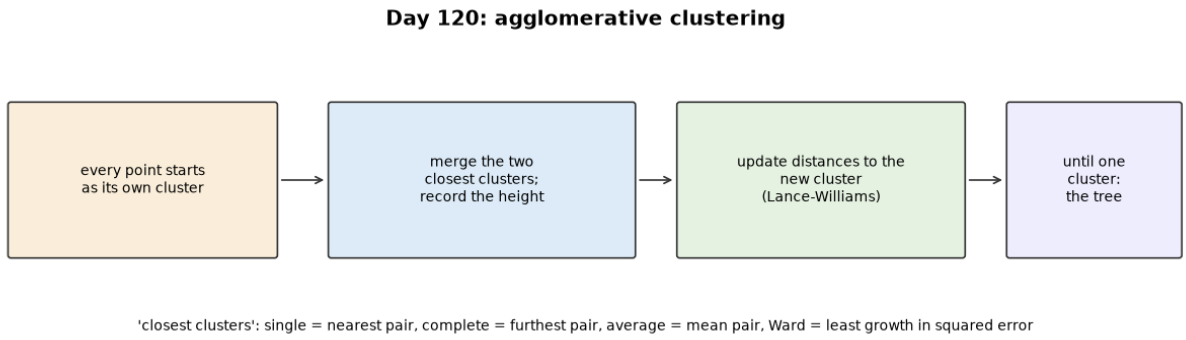
Agglomerative clustering starts with every point as its own cluster and repeatedly merges the two closest clusters until one is left. The record of merges, with the distance at which each happened, is a tree; drawn, it is a **dendrogram**. Cutting it at any height gives a clustering.

Everything depends on the **linkage**, the meaning of 'distance between clusters': **single** uses the nearest pair of points, **complete** the furthest pair, **average** the mean over all pairs, and **Ward** the growth in squared error the merge would cause.

REAL-WORLD SCENARIO

A family tree read upside down, or the tree of life: species merge into genera, genera into families. Biologists can stop at any level and still have a sensible grouping; nobody has to decide in advance how many groups there are.

SYSTEM DIAGRAM



Merge, update the distances, repeat until one cluster is left.

Today's part for the week's build

`day120_exercise.py` is the merge loop, the cut and the cophenetic heights, checked by hand on five points. Its real output:

ACTUAL OUTPUT

```
single linkage on 0, 1, 3, 7, 8.5: merges [(0, 1, 1.0), (3, 4, 1.5), (2, 5, 2.0), (6, 7, 4.0)]
by hand: 0-1 at 1, 7-8.5 at 1.5, then 3 joins {0,1} at 2, and the two groups meet at 4 (3 to 7)
cut into 2: [1, 1, 1, 0, 0]; into 5: [0, 1, 2, 3, 4]; into 1: [0, 0, 0, 0, 0]
cophenetic height between 0 and 8.5: 4.0 (the last merge); between 0 and 3: 2.0
merge heights never decrease: single True, complete True, average True, ward True
```

Data: iris and wine, both standardised, and the digits as raw pixels, all bundled with scikit-learn; their labels are used only to judge. SciPy's `linkage` is an outside check.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: five points

DEFINITION — start here, no prior knowledge assumed

Points 0, 1, 3, 7 and 8.5 on a line, with single and complete linkage. Each row of the history is (cluster, cluster, height, size); the first merge creates cluster 5, the next cluster 6, and so on.

REAL-WORLD SCENARIO

Five houses along a road, joined into neighbourhoods: first the two closest, then the next closest pair of groups.

MATHEMATICS

single: $d(A, B) = \min$ over pairs $|a - b|$; complete: max over pairs; average: mean over pairs

Lance-Williams, after merging i and j into one cluster: $d(i \cup j, k) = \min(d_{ik}, d_{jk})$ for single, max for complete

IMPLEMENTATION

```
def agglomerate(X, method):
    n = len(X); D = np.sqrt(np.maximum(sqdist(X, X), 0))
    if method == "ward":
        D = D ** 2
    D[np.arange(n), np.arange(n)] = np.inf
    size = np.ones(n); ids = np.arange(n); alive = np.ones(n, bool); Z = []
    for step in range(n - 1):
        M = np.where(alive[:, None] & alive[None, :], D, np.inf)
        i, j = np.unravel_index(np.argmin(M), M.shape); i, j = min(i, j), max(i, j)
        d = M[i, j]; ni, nj = size[i], size[j]
        Z.append((min(ids[i], ids[j]), max(ids[i], ids[j]), np.sqrt(d) if method == "ward" else d,
            ni + nj))
        nk = size
        if method == "single":
            new = np.minimum(D[i], D[j])
        elif method == "complete":
            new = np.maximum(D[i], D[j])
        elif method == "average":
            new = (ni * D[i] + nj * D[j]) / (ni + nj)
        else:
            new = ((ni + nk) * D[i] + (nj + nk) * D[j] - nk * D[i, j]) / (ni + nj + nk)
        D[i, :] = new; D[:, i] = new; D[i, i] = np.inf
        alive[j] = False; size[i] = ni + nj; ids[i] = n + step
    return np.array(Z)
```

ACTUAL OUTPUT

```
points 0, 1, 3, 7, 8.5; each row of the history is (cluster, cluster, height, size); clusters 5,
    6, ... are merges
  single   : (0, 1) at 1.00; (3, 4) at 1.50; (2, 5) at 2.00; (6, 7) at 4.00
  complete : (0, 1) at 1.00; (3, 4) at 1.50; (2, 5) at 3.00; (6, 7) at 8.50
by hand, single: 0-1 at 1; then {0,1}-3 at 2 (nearest pair 1-3); 7-8.5 at 1.5 comes before that;
    the two groups at 4 (3 to 7)
complete uses the FURTHEST pair: {0,1}-3 at 3 (0 to 3), and the last merge at 8.5 (0 to 8.5)
Actual output from this lesson's run.
```

What it means

Both linkages make the same first two merges (0 with 1 at 1, 7 with 8.5 at 1.5), then differ. Single linkage joins 3 to {0, 1} at 2.00, its nearest pair, and ends at 4.00, the gap between 3 and 7. Complete linkage measures the furthest pair: 3 joins at 3.00 (0 to 3), and the last merge is at 8.50, the whole width. The update rule never recomputes distances from points: it builds each new row from two old ones, which is why one function serves all four linkages.

WHY CHOOSE THIS?

One loop and one line per linkage.

WHY NOT THIS? (TRADEOFFS)

Every merge searches the whole table; section 7 measures what that costs.

2. Correctness: against SciPy

DEFINITION — start here, no prior knowledge assumed

All four linkages on iris, compared with SciPy's merge heights and its cophenetic correlation (defined in section 3).

REAL-WORLD SCENARIO

Checking a hand-drawn family tree against the parish records.

ACTUAL OUTPUT

```
  single   : merge heights, largest gap 7.0e-15; cophenetic correlation 0.830005 against SciPy's 0.830005
  complete : merge heights, largest gap 2.4e-02; cophenetic correlation 0.751460 against SciPy's 0.751459
  average  : merge heights, largest gap 7.0e-15; cophenetic correlation 0.854361 against SciPy's 0.854361
  ward     : merge heights, largest gap 7.0e-15; cophenetic correlation 0.822630 against SciPy's 0.822630
(heights are compared in sorted order: when two merges tie, the two versions may take them in a different
    order)
iris has 1631 tied distances (iris measurements are rounded to 0.1 cm); with ties, complete linkage can build
    a different tree
the same check after adding noise of size 1e-6 to break the ties: largest gap single 1.3e-10, complete
    1.3e-10, average 1.3e-10, ward 1.3e-10
Actual output from this lesson's run.
```

What it means

Single, average and Ward match SciPy to 7.0e-15. Complete linkage did not: a gap of 2.4e-02, and correlations that differ in the sixth decimal (0.751460 against 0.751459). The script checked why. Iris has 1631 tied distances, because its measurements are rounded to 0.1 cm, and when two merges tie, the two programs may take them in a different order. For complete linkage that changes later distances, and so the tree. Adding noise of size 10^{-6} breaks every tie, and then all four agree to 1.3e-10.

WHY CHOOSE THIS?

A real explanation for a real mismatch, tested rather than assumed.

WHY NOT THIS? (TRADEOFFS)

With ties, 'the' tree is not unique; neither program is wrong.

3. Four linkages on three datasets

DEFINITION — start here, no prior knowledge assumed

Each tree is cut into the true number of groups and scored with Day 119's ARI. The **cophenetic correlation** asks how faithfully the tree keeps the original distances: the height at which two points first share a cluster, compared with how far apart they really are.

REAL-WORLD SCENARIO

Judging a map by whether places that look close on it are really close.

MATHEMATICS

cophenetic(a, b) = the height of the merge that first puts a and b in one cluster; cophenetic correlation = corr(distances, cophenetic heights)

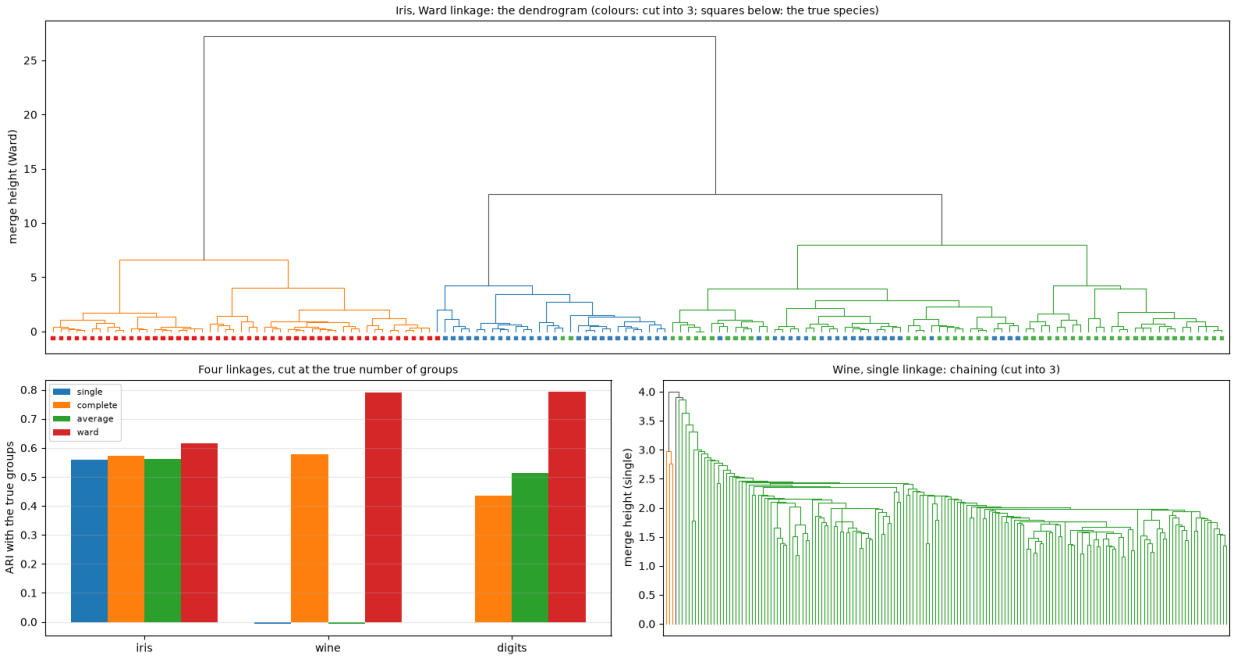
IMPLEMENTATION

```
def cut(Z, n, k):
    parent = np.arange(2 * n - 1)
    for s, (a, b, _, _) in enumerate(Z[:n - k]):
        parent[int(a)] = parent[int(b)] = n + s
    def root(x):
        while parent[x] != x:
            x = parent[x]
        return x
    r = np.array([root(i) for i in range(n)])
    return np.unique(r, return_inverse=True)[1]
def cophenetic(Z, n):
    members = {i: [i] for i in range(n)}; C = np.zeros((n, n))
    for s, (a, b, h, _) in enumerate(Z):
        A, B = members.pop(int(a)), members.pop(int(b))
        C[np.ix_(A, B)] = h; C[np.ix_(B, A)] = h
```

```
        members[n + s] = A + B
    return C
def coph_corr(X, Z):
    n = len(X); D = np.sqrt(np.maximum(sqdist(X, X), 0)); C = cophenetic(Z, n); iu =
        np.triu_indices(n, 1)
    return float(np.corrcoef(D[iu], C[iu])[0, 1])
```

ACTUAL OUTPUT

```
data    linkage        ARI                          cluster sizes  cophenetic
iris    single      0.5584                           [100, 49, 1]      0.8300
iris    complete    0.5726                           [77, 49, 24]      0.7515
iris    average     0.5621                            [97, 50, 3]      0.8544
iris    ward        0.6153                           [71, 49, 30]      0.8226
wine    single     -0.0068                            [174, 3, 1]      0.5436
wine    complete    0.5771                           [69, 58, 51]      0.5917
wine    average    -0.0054                            [174, 3, 1]      0.7591
wine    ward        0.7899                           [64, 58, 56]      0.6623
digits  single      0.0000                [1788, 1, 1, 1, '...']      0.4220
digits  complete    0.4356           [398, 260, 237, 228, '...']      0.5388
digits  average     0.5142           [480, 363, 248, 193, '...']      0.6614
digits  ward        0.7940           [317, 197, 196, 191, '...']      0.5663
Actual output from this lesson's run.
```



Top: the iris Ward dendrogram, coloured by the cut into 3, with the true species as squares under each leaf. Bottom left: ARI for each linkage. Bottom right: single linkage on wine, a staircase of one-point merges.

What it means

Ward is best on all three: 0.6153, 0.7899 and 0.7940. Single and average linkage fail on wine, leaving cluster sizes [174, 3, 1]: one cluster plus two scraps of outliers. The cophenetic correlation does not track usefulness: average linkage keeps the distances best on every dataset (0.8544 on iris), while Ward, the most useful here, does not. A tree can preserve distances well and still cut badly. In the dendrogram, setosa (red squares) is a clean branch, while versicolor and virginica interleave under the other two.

WHY CHOOSE THIS?

One tree serves every k, and the dendrogram shows the structure at every scale.

WHY NOT THIS? (TRADEOFFS)

Each linkage builds a different tree; the choice matters as much as k.

4. Single linkage's chaining

DEFINITION — start here, no prior knowledge assumed

Single linkage merges two clusters if *any* two of their points are close, so a chain of near neighbours can join everything.

REAL-WORLD SCENARIO

Rumours spreading through a crowd: everyone is connected to everyone through someone.

ACTUAL OUTPUT

```
  cut into  2 clusters: the largest holds 1796 of 1797 images; clusters of a single image: 1
  cut into 10 clusters: the largest holds 1788 of 1797 images; clusters of a single image: 9
  cut into 50 clusters: the largest holds 1731 of 1797 images; clusters of a single image: 44
single linkage merges through any chain of near neighbours, so one cluster swallows almost
    everything and outliers stay alone
Actual output from this lesson's run.
```

What it means

Cut into 10 clusters, digits gives one cluster of 1788 images and 9 clusters of a single image. Even at 50 clusters the largest holds 1731. On data with outliers, single linkage finds the outliers, not the groups. Its strength is the opposite case: long, thin, well-separated shapes, which Ward and k-means would cut.

WHY CHOOSE THIS?

Finds groups of any shape, if they are separated by clear gaps.

WHY NOT THIS? (TRADEOFFS)

Any bridge of noise points merges two groups into one.

5. Where to cut

DEFINITION — start here, no prior knowledge assumed

A common rule: cut where the gap between successive merge heights is largest, among the last 15 merges. A first version of this rule had an off-by-one error, reporting one cluster too few, and was fixed before this run.

REAL-WORLD SCENARIO

Reading a family tree for the point where branches stop being close relatives.

ACTUAL OUTPUT

```
  iris   : last five merge heights [4.24, 6.61, 8.0, 12.64, 27.25]; the biggest gap suggests k = 2
      (true 3); ARI 0.5438
  wine   : last five merge heights [11.72, 12.23, 12.57, 27.65, 35.4]; the biggest gap suggests k
      = 3 (true 3); ARI 0.7899
  digits : last five merge heights [436.61, 450.9, 488.62, 536.32, 691.96]; the biggest gap
      suggests k = 2 (true 10); ARI 0.1458
Actual output from this lesson's run.
```

What it means

The rule finds 3 on wine (ARI 0.7899) but 2 on iris and on digits. On iris the jump to the last merge (12.64 to 27.25) dominates: setosa really is far from the other two species, so 2 is a defensible answer (0.5438). On digits it suggests k = 2 against 10 true groups (0.1458). Like the elbow on Day 119, it reads the biggest structure first, which is not always the one wanted.

WHY CHOOSE THIS?

No k needed in advance: the tree suggests one.

WHY NOT THIS? (TRADEOFFS)

It finds the largest gap, not necessarily the grouping you need.

6. Ward against k-means

DEFINITION — start here, no prior knowledge assumed

Both minimise the within-cluster squared distance. Ward does it greedily, merge by merge, and never undoes a merge; k-means (Day 119) keeps moving points between clusters.

REAL-WORLD SCENARIO

Two ways to pack a removal van: box by box, never repacking, or loading everything and then swapping things around.

ACTUAL OUTPUT

```
  iris   : inertia Ward 148.9, k-means 139.8; ARI with truth Ward 0.6153, k-means 0.6201; agreement between
      the two 0.6255
  wine   : inertia Ward 1305.0, k-means 1277.9; ARI with truth Ward 0.7899, k-means 0.8975; agreement between
      the two 0.8530
  digits : inertia Ward 1191606.8, k-means 1167803.8; ARI with truth Ward 0.7940, k-means 0.6525; agreement
      between the two 0.8179
Ward builds greedily, merge by merge, and never revisits a merge; k-means keeps moving points, so it usually
    reaches a lower inertia for one k
Actual output from this lesson's run.
```

What it means

k-means reaches a lower inertia every time (139.8 against 148.9 on iris), as expected from an algorithm that keeps improving. Yet a lower inertia is not a better answer: on digits Ward agrees with the true digits at 0.7940, k-means at 0.6525. On wine it is the other way (0.8975 for k-means). The two agree with each other at 0.6255 to 0.8530: similar, not the same.

WHY CHOOSE THIS?

Ward gives a whole tree, with no random start.

WHY NOT THIS? (TRADEOFFS)

It cannot fix an early mistake, and the tree costs far more to build than k-means.

7. The cost of a whole tree

DEFINITION — start here, no prior knowledge assumed

The time to build Ward's tree on growing slices of the digits, and the size of the distance table.

REAL-WORLD SCENARIO

A tournament where every round compares every remaining pair of players.

ACTUAL OUTPUT

```
    200 images: 0.01 s, and a distance table of 0.3 MB
    400 images: 0.16 s, and a distance table of 1.3 MB
    800 images: 1.27 s, and a distance table of 5.1 MB
   1600 images: 11.56 s, and a distance table of 20.5 MB
this simple version searches the whole table at every merge: time grows with n cubed, memory with
    n squared (within this run)
Actual output from this lesson's run.
```

What it means

Doubling from 800 to 1600 images took the time from 1.27 s to 11.56 s, about 9 times, near the 8 that n^3 predicts. Each of the n – 1 merges searches the whole n × n table. The table itself grows with n^2: about 20 MB at 1600 points, and 80 GB at 100,000. Libraries use cleverer searches, but hierarchical clustering remains a method for thousands of points, not millions.

WHY CHOOSE THIS?

Exact, simple and deterministic.

WHY NOT THIS? (TRADEOFFS)

Memory grows with n squared, which rules out very large datasets.

Interview preparation — technical questions

Q: How does agglomerative clustering work?

A: Start with every point alone; merge the two closest clusters; repeat until one is left; the merges form a dendrogram.

Q: Which linkage would you try first?

A: Ward: the best on all three datasets here (0.6153, 0.7899, 0.7940).

Q: How do you choose the number of clusters?

A: Cut the tree; the biggest-gap rule found 3 on wine but 2 on iris and digits, so check with a purpose in mind.

Q: Why might two correct programs give different trees?

A: Tied distances: iris has 1631, and complete linkage then depends on the order of ties.

Q: Hierarchical clustering or k-means?

A: A tree and no random start, against speed: k-means scales to millions of rows; a full tree does not.

Interview preparation — theoretical questions

Q: What is the Lance-Williams update?

A: A rule for the distance from a merged cluster to every other one, from the old distances alone.

Q: What is chaining?

A: Single linkage joining groups through a chain of close points: one cluster of 1788 of 1797 digits.

Q: What does the cophenetic correlation measure?

A: How well the tree's merge heights keep the original distances; it did not predict which linkage cut best.

Q: Why does Ward relate to k-means?

A: Both minimise within-cluster squared error; Ward greedily, k-means by moving points (139.8 against 148.9 on iris).

Key takeaway and what's next

> **KEY TAKEAWAY**
> A dendrogram answers 'how many groups?' at every scale at once, and the linkage decides what a group is. Ward's squared-error rule suited all three datasets; single linkage chained everything together. The tree is exact and needs no random start, but it costs n squared memory, and choosing where to cut is still a judgement.

- Ward: ARI 0.6153 (iris), 0.7899 (wine), 0.7940 (digits); k-means on digits 0.6525.
- Single linkage on digits, cut into 10: one cluster of 1788.
- SciPy matched to 7.0e-15; complete linkage differed only through 1631 tied distances.
- Biggest-gap cut: 3 on wine, 2 on iris and digits.

Suggested watch order, if starting from zero

- 1. Hierarchical clustering - explained (TileStats) — building a dendrogram by hand and cutting it.
- 2. Hierarchical Clustering & Dendrograms — Single, Complete, Average & Ward Linkage (Naive Statistician) — the four linkages side by side.

Day 121 preview

- **DBSCAN from scratch**: clusters as dense regions of any shape, with points in sparse regions left as noise.